

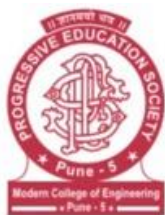
RESEARCH PROJECT
ON
RELAY: A MULTI-TENANT RETRIEVAL-AUGMENTED
GENERATION
PLATFORM FOR DOCUMENT-GROUNDED CHATBOTS

BY
Aaditi Choramle
Chitrang Choudhari
Pratik Kamble
Sahil Dhanavade

Under the Guidance of
Dr. Swati D. Ghule

MASTER OF COMPUTER APPLICATION
P.E.S'S MODERN COLLEGE OF ENGINEERING
PUNE – 411 005
(An Autonomous Institute Affiliated to Savitribai Phule Pune University)

2026-27



Progressive Education Society's
Modern College of Engineering
(An Autonomous Institute Affiliated to
Savitribai Phule Pune University)
MCA Department

CERTIFICATE

This is to certify that **AADITI CHORAMLE, CHITRANG CHOUDHARI, PRATIK KAMBLE and SAHIL DHANAVADE** of Master in computer Application have successfully completed the Research Project work titled '**RELAY: A MULTI-TENANT RETRIEVAL-AUGMENTED GENERATION PLATFORM FOR DOCUMENT-GROUNDED CHATBOTS**' during the academic year 2026-27. This report is submitted as partial fulfillment of the requirement of degree in MCA Engineering of Modern College of Engineering.

External Examiner:

**Prof. Dr. Mrs. K. R.
Joshi
Principal**

**Prof. Dr. Shivani A.
Budhkar
Head of Department**

**Dr. Swati D. Ghule
Project Guide**

ACKNOWLEDGEMENT

We take this opportunity to express our sincere gratitude to everyone who supported us during the completion of this research project.

We are thankful to **Prof. Dr. Mrs. K. R. Joshi**, Principal, P.E.S'S Modern College of Engineering, Pune, for providing the academic environment and facilities required to carry out this work.

We express our deep sense of gratitude to **Prof. Dr. Shivani A. Budhkar**, Head of the MCA Department, for her constant encouragement and for creating a research-oriented atmosphere in the department.

We are especially indebted to our project guide, **Dr. Swati D. Ghule**, for valuable guidance, timely suggestions, and continuous support throughout the design, implementation, and documentation of this project. The discussions on retrieval-augmented generation, multi-tenant isolation, and system architecture helped us convert a research idea into a working platform.

We also thank the faculty members of the MCA Department for their teaching in software engineering, database systems, and emerging technologies, which formed the foundation of this work. We acknowledge the open-source communities behind Next.js, Auth.js, Pinecone, and OpenRouter, whose platforms made a production-grade RAG system feasible within an academic timeline.

Finally, we thank our families and classmates for their patience and moral support during the project period.

Sign of students: _____

Name of students: Aaditi Choramle, Chitrang Choudhari, Pratik Kamble, Sahil Dhanavade

Roll no.: _____

ABSTRACT

Large Language Models (LLMs) generate fluent answers but are prone to hallucination and cannot reliably restrict themselves to an organisation’s private documents. Retrieval-Augmented Generation (RAG) addresses this by retrieving relevant text chunks from a knowledge base and conditioning the LLM on that evidence. Existing commercial chatbot builders are closed, costly, and offer limited control over tenant isolation. Typical open-source tutorials are single-tenant prototypes that do not expose a secure public embed API.

This project presents **Relay**, a multi-tenant SaaS platform on which a signed-in owner can create independent chatbots, upload PDF, DOCX, and TXT documents, and obtain grounded answers with source citations. Documents are parsed, split with recursive character chunking (chunk size 800, overlap 150), embedded with Pinecone’s hosted **multilingual-e5-large** model, and stored in a shared serverless vector index. Isolation is enforced by a server-side metadata filter on `chatbot_id`, so Chatbot A cannot retrieve Chatbot B’s chunks. Query answering uses the same embedding model with `inputType: query`, cosine top-k retrieval, and answer synthesis through OpenRouter (primary model Llama 3.3 70B Instruct, with free-tier fallbacks). The dashboard and the public widget share one RAG pipeline. The widget is published as the npm package `relay-chat-widget`, authenticated by a per-chatbot public API key (`pk_live_...`) and an origin allow-list.

The system is implemented as a single Next.js 15 service with Auth.js (Google OAuth, JWT sessions), MongoDB/Mongoose for users and chatbot configuration, and Jest unit tests for chunking, models, registration, and chatbot APIs. It is deployed at <https://relayy-dun.vercel.app>. The work demonstrates that a complete multi-tenant, document-grounded chatbot product can be built without a separate Python embedding microservice, while preserving tenant isolation and citation-backed answers.

Keywords: Retrieval-Augmented Generation, Multi-tenant SaaS, Vector search, Pinecone, Large Language Models, Embeddable chatbot widget, Next.js

CONTENTS

Certificate.....	ii
Acknowledgement	iii
Abstract.....	iv
List of Abbreviations	vi
List of Figures	vii
List of Tables	viii
1 Introduction, Aims, Motivation and Objectives	1
2 Literature Survey	4
3 Research Gap	7
4 Problem Statement / Definition	9
5 Proposed System / Proposed Methodology	11
6 Diagrams (Architecture, Flowcharts, UML, ERD).....	18
7 Project Requirement Specification	24
8 System Implementation and Code Documentation.....	27
9 Result / Outcome / Experimental Result.....	31
10 System Testing.....	34
11 GUI / Screenshots	36
12 Conclusions.....	40
13 Limitations and Future Scope	41
14 References.....	43
15 Plagiarism Report.....	45

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
DPR	Dense Passage Retrieval
ERD	Entity Relationship Diagram
JWT	JSON Web Token
LLM	Large Language Model
MCA	Master of Computer Application
NLP	Natural Language Processing
OAuth	Open Authorization
ODM	Object Document Mapper
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
SaaS	Software as a Service
SRS	Software Requirement Specification
UML	Unified Modeling Language
UUID	Universally Unique Identifier

LIST OF FIGURES

Figure No.	Caption
Fig. 1.1	High-level context of Relay in a customer website
Fig. 5.1	Proposed system architecture
Fig. 5.2	Document ingestion pipeline
Fig. 5.3	Query-time RAG pipeline
Fig. 6.1	System architecture diagram
Fig. 6.2	Use-case diagram
Fig. 6.3	Sequence diagram — document upload
Fig. 6.4	Sequence diagram — dashboard Q&A
Fig. 6.5	Sequence diagram — public widget chat
Fig. 6.6	Entity-relationship diagram
Fig. 6.7	Class diagram (core domain)
Fig. 6.8	Activity diagram of RAG answering
Fig. 11.1	Landing page
Fig. 11.2	Google sign-in page
Fig. 11.3	Owner dashboard showing four chatbots (raj, ram, sample, sahil)
Fig. 11.4	Create chatbot modal
Fig. 11.5	Chatbot workspace — knowledge-base upload pane
Fig. 11.6	Chatbot workspace — answer to “tell me the skills of sahil”
Fig. 11.7	Embed & API — public key, origins, widget appearance
Fig. 11.8	Embed & API — script-tag and npm install snippets

LIST OF TABLES

Table No.	Caption
Table 2.1	Comparison of related systems
Table 7.1	Hardware requirements
Table 7.2	Software requirements
Table 7.3	Functional requirements
Table 7.4	Non-functional requirements
Table 8.1	API route catalogue
Table 8.2	Pinecone vector metadata schema
Table 9.1	Qualitative RAG outcomes
Table 9.2	Sample dashboard Q&A on chatbot <i>sahil</i>
Table 10.1	Unit test summary
Table 10.2	API test cases
Table 10.3	Isolation and security test cases

1. INTRODUCTION AND AIMS / MOTIVATION AND OBJECTIVES

1.1 Introduction

Organisations accumulate knowledge in PDFs, policy manuals, product guides, and internal notes. Employees and website visitors repeatedly ask questions whose answers already exist in those files, yet finding the right paragraph is slow. Conventional keyword search fails when the user paraphrases. Fine-tuning a large language model on private data is expensive, slow to update, and still does not prove *where* an answer came from.

Retrieval-Augmented Generation (RAG), introduced by Lewis et al. [1], combines a parametric generator (an LLM) with a non-parametric memory (a vector index of documents). At question time the system embeds the query, retrieves the most similar text chunks, and asks the LLM to answer using only that context. This reduces hallucination, supports citations, and allows the knowledge base to be updated by uploading a new file rather than retraining a model.

Relay applies this idea as a **multi-tenant product**. Each signed-in user may own many chatbots. Each chatbot has its own documents, API key, origin allow-list, and widget appearance. Website visitors never see the owner's dashboard; they talk to a floating chat button published as `relay-chat-widget`. Both paths call the same server-side answer function so that behaviour is consistent.

The platform is a single Next.js 15 application. MongoDB stores users and chatbot configuration. Pinecone stores embeddings and performs hosted inference, which removes the need for a Python embedding service. OpenRouter supplies LLM completions, including free-tier models suitable for academic deployment.

1.2 Motivation

The motivation for this research project is both practical and academic.

Practical motivation. Small businesses, college departments, and independent developers need a document chatbot they can embed on a site without hiring an ML team. Commercial tools such as Chatbase or CustomGPT charge subscription fees and hide

retrieval internals. Building a one-off LangChain notebook does not yield login, tenancy, CORS, or an npm widget.

Academic motivation. RAG literature focuses on retrieval quality and generation faithfulness [1][3]. Multi-tenant software engineering literature focuses on isolation of compute and data [12]. Few teaching-scale systems show how both concerns meet in one codebase: a shared vector index, a metadata filter that must never be omitted, a public key that must not grant write access, and a widget that must obey CORS. Implementing and documenting that combination is a useful research-oriented engineering contribution for an MCA dissertation.

Cost motivation. Generating embeddings through a separate OpenAI embedding endpoint consumes paid tokens. Pinecone hosted inference (`pc.inference.embed`) produces vectors with the same model that the index expects, in batches of at most 96 inputs, without a second vendor for embeddings. LLM answers go through OpenRouter so that free-tier models can be used with automatic fallbacks when a model is rate-limited.

1.3 Aim

To design, implement, and deploy a production-ready multi-tenant platform on which users can create isolated, document-grounded AI chatbots and embed them on third-party websites, with answers generated by retrieval-augmented generation and accompanied by source citations.

1.4 Objectives

1. To study RAG, dense retrieval, multi-tenant SaaS isolation, and related chatbot platforms, and to identify the research gap this work addresses.
2. To design a single-service architecture in which authentication, document ingestion, vector search, LLM answering, and the public widget API live in one Next.js application.
3. To implement secure owner authentication using Auth.js v5 with Google OAuth and JWT sessions, and to authorise every dashboard API by matching `chatbot.userId` to the session user.

4. To implement document ingestion for PDF, DOCX, and TXT: text extraction, recursive chunking with overlap, hosted embedding, and upsert into Pinecone with `chatbot_id` metadata.
5. To implement query-time RAG: query embedding, top-k cosine search filtered by `chatbot_id`, and grounded generation via OpenRouter with fallback models and a raw-context fallback if the LLM is unavailable.
6. To expose a public, CORS-enabled chat API authenticated by a per-chatbot `pk_live_` key and an origin allow-list, without revealing internal chatbot UUIDs to the customer site.
7. To publish an embeddable widget (`relay-chat-widget`) that loads appearance from the dashboard and sends questions to the public API.
8. To verify isolation, chunking, models, and API behaviour with automated tests, and to deploy the system for demonstration.

1.5 Scope

In scope

- Owner sign-in, chatbot CRUD, document upload (PDF / DOCX / TXT), dashboard chat with citations.
- Multi-tenant isolation on a shared Pinecone index via metadata filters.
- Public widget API and npm/script-tag embed.
- Widget appearance (position, colour, welcome message) stored per chatbot.
- Chunked upload for files larger than approximately 3.5 MB (Vercel body-size limit).

Out of scope

- Fine-tuning or training of embedding/LLM weights.
- Conversation memory across sessions, user analytics dashboards, and billing.
- Image, table, or OCR-heavy PDF understanding.
- Per-tenant physical Pinecone indexes (logical isolation is used instead).

2. LITERATURE SURVEY

2.1 Transformer language models and the hallucination problem

Vaswani et al. [6] introduced the Transformer, which is the backbone of modern LLMs. Devlin et al. [9] showed that bidirectional pre-training (BERT) yields strong text representations. Subsequent decoder-only models such as Llama 2 [10] and Llama 3 [11] generate fluent open-ended text. OpenAI's GPT-4 technical report [19] documents strong general capability but does not solve the problem of answering *only* from a private corpus.

Ji et al. [4] survey hallucination in natural language generation: models produce plausible statements that are not supported by source text. For enterprise assistants this is unacceptable. A leave-policy bot that invents extra holidays is worse than a bot that admits it does not know.

2.2 Dense retrieval

Sparse lexical methods (BM25) fail when query vocabulary differs from document vocabulary. Karpukhin et al. [2] proposed Dense Passage Retrieval (DPR): questions and passages are encoded into the same vector space and nearest neighbours are retrieved. Reimers and Gurevych [7] (Sentence-BERT) made semantically meaningful sentence embeddings practical. Johnson et al. [8] demonstrated billion-scale approximate nearest neighbour search (FAISS), which industrial vector databases extend.

Wang et al. [5] released multilingual E5 embeddings. The **multilingual-e5-large** model used in this project has 24 layers and 1024-dimensional vectors, supports about 100 languages, and distinguishes passage vs query input types. Relay therefore embeds documents with `inputType: passage` and questions with `inputType: query`, matching the model's training.

2.3 Retrieval-Augmented Generation

Lewis et al. [1] defined RAG as a generator conditioned on passages retrieved from a non-parametric index (originally Wikipedia). The parametric memory can stay frozen while the index is updated. Izacard and Grave [21] (Fusion-in-Decoder) showed that concatenating many retrieved passages into a seq2seq encoder improves open-domain QA. Gao et al. [3] survey RAG for LLMs and distinguish Naive RAG (index–retrieve–generate), Advanced

RAG (pre/post retrieval optimisation), and Modular RAG. Relay implements a production Naive/Advanced hybrid: recursive chunking and overlap (pre-retrieval), metadata filtering and top-k (retrieval), and a strict system prompt plus reasoning-leak detection (post-generation).

Asai et al. [20] (Self-RAG) add reflection tokens so the model can decide when to retrieve. That line of work is more complex than required here; Relay always retrieves for a user question and instructs the LLM to refuse when context is insufficient.

2.4 Vector databases and hosted inference

Pinecone [13] provides a managed vector index with metadata filters and, more recently, hosted inference so that embedding happens inside the same vendor as storage. This avoids dimension mismatch between a local embedding model and the index. OpenRouter [14] exposes many LLMs behind an OpenAI-compatible HTTP API and supports a `models` fallback array, which Relay uses when the primary free model is unavailable.

2.5 Orchestration frameworks and commercial chatbot builders

LangChain [18] popularised RAG chains in Python. Most tutorials wire one folder of files to one bot. They rarely include OAuth, per-tenant API keys, origin allow-lists, or an npm widget. Commercial products (Chatbase, SiteGPT, CustomGPT, Intercom Fin) offer hosted RAG chatbots as a service. They validate market demand but are closed-source, which limits academic inspection of isolation and prompting.

2.6 Multi-tenant software architecture

Krebs, Momm and Kounev [12] discuss isolation, customisation, and scalability in multi-tenant SaaS. Three isolation levels are common: separate database per tenant, separate schema per tenant, and shared schema with a tenant identifier. Relay uses the third pattern twice:

- MongoDB: `Chatbot.userId` ties bots to owners; dashboard routes compare this with the `JWT session.user.id`.
- Pinecone: every vector carries `chatbot_id`; every query includes `filter: { chatbot_id: { $eq: chatbotId } }`.

A missing filter would be a data-leak bug. The implementation therefore centralises answering in one function (`answerChatbotQuery`) that always receives an explicit chatbot id from an already-authorised caller.

2.7 Web authentication, CORS, and public widgets

Auth.js (NextAuth) [15] implements OAuth 2.0 / OpenID Connect for Next.js. Relay uses Google as the identity provider and JWT session strategy so that serverless instances need no shared session store. Fielding’s REST dissertation [22] informs the resource-oriented API design. Browser widgets on other origins require CORS; Relay authenticates the widget with a public API key and enforces `allowedOrigins` on the actual GET/POST (preflight is permissive because the key is not sent on OPTIONS). OWASP Top 10 [23] motivates hashed secrets, ownership checks, and avoiding leakage of stack traces.

2.8 Comparative summary

Table 2.1 Comparison of related systems

System	Multi-tenant	Document RAG	Embeddable widget	Open internals	Hosted embeddings (no extra Python service)
LangChain tutorial bot	No	Yes	Rarely	Yes	No (local/OpenAI)
Chatbase / CustomGPT	Yes	Yes	Yes	No	Vendor-managed
Fine-tuned LLM only	N/A	Weak (parametric)	N/A	Varies	N/A
Relay (this work)	Yes (metadata filter)	Yes (Pinecone + OpenRouter)	Yes (<code>relay-chat-widget</code>)	Yes (this report + code)	Yes (Pinecone inference)

3. RESEARCH GAP

From the survey, the following gaps are identified:

Gap 1 — Isolation is under-specified in educational RAG systems.

Published notebooks retrieve from a single index with no tenant key. In a shared index, omitting a metadata filter silently mixes tenants. Academic reports often draw a “namespace per bot” box that is not what the code does. This work documents and implements **logical isolation on a shared index**, which is the pattern actually used in many SaaS vector deployments.

Gap 2 — Closed commercial platforms versus incomplete open prototypes.

Industry tools prove that document chatbots are valuable but cannot be examined for a dissertation. Open prototypes usually stop at a chat UI in localhost, without Google login, public API keys, origin allow-lists, or a published npm widget.

Gap 3 — Split-brain architecture (Next.js + Python embedder).

Many student projects run FastAPI for embeddings and Next.js for UI. That doubles deployment, environment variables, and failure modes. Pinecone hosted inference makes a **single Node.js service** sufficient. This gap is engineering research: can a full RAG SaaS avoid a Python sidecar without sacrificing embedding quality?

Gap 4 — Two answer paths that drift apart.

If the dashboard and the website widget each implement retrieval, prompts diverge and bugs appear in only one path. Relay forces both through `answerChatbotQuery`. The research interest is architectural: a single pipeline with two authentication facades (session vs API key).

Gap 5 — Free-tier LLM brittleness.

Academic projects often hard-code one model and fail when the free endpoint is rate-limited or leaks chain-of-thought. Relay treats that as a first-class problem: OpenRouter model fallbacks, detection of leaked reasoning traces, and a deterministic raw-chunk fallback so the user still sees source text.

Gap 6 — Public keys are not secrets.

Widget keys live in browser JavaScript. The gap in many designs is treating them like server secrets. Relay designs them as *public identifiers* (`pk_live_`) that only permit chat, never upload or delete, and that are further bound by CORS origins.

This project occupies the gap between a research RAG pipeline and a small multi-tenant product: inspectable, deployable, and isolation-aware.

4. PROBLEM STATEMENT / DEFINITION

4.1 Problem statement

Organisations need website and internal assistants that answer questions **only** from their own documents, with visible sources, while multiple organisations share the same hosted platform. Existing solutions are either (a) closed commercial SaaS, or (b) single-tenant prototypes that do not provide authentication, tenant isolation on a shared vector index, or a safe public embed API.

4.2 Formal definition

Let (U) be the set of users and (C) the set of chatbots. Each chatbot $(c \in C)$ has an owner $(owner(c) \in U)$, a document collection (D_c) , and a public key (k_c) .

Ingestion. For each document $(d \in D_c)$, extract text, split into chunks $(t_{c,i})$ of maximum length 800 characters with overlap 150, compute embeddings $(v_{c,i} = E_{\{passage\}}(t_{c,i}))$, and store $((v_{c,i}, metadata_{c,i}))$ where $(metadata_{c,i})$ contains $chatbot_id = c$.

Query. Given question (q) issued either by $(owner(c))$ (session) or by a widget presenting (k_c) from an allowed origin:

1. $(v_q = E_{\{query\}}(q))$
2. Retrieve top- (k) neighbours of (v_q) subject to filter $chatbot_id = c$
3. Generate answer $(a = LLM(q, t_{c,i}))$ under a prompt that forbids use of parametric knowledge outside the chunks
4. Return (a) together with source names and similarity scores

Isolation invariant. For any query on chatbot (c) , the retrieved set must be a subset of chunks whose metadata chatbot id equals (c) . No other chatbot's text may appear in the LLM context.

Authorisation invariant. Dashboard mutations require `session.user.id = owner(c)`. Public chat requires a valid (k_c) and an allowed Origin. Possession of (k_c) must not allow document upload or chatbot deletion.

4.3 Research questions

1. Can tenant isolation on a shared Pinecone index be enforced solely by server-side metadata filters, without a separate index per chatbot?
2. Can dashboard chat and third-party widget chat share one RAG implementation without leaking owner credentials to the browser widget?
3. Can a complete RAG SaaS run as one Next.js process using hosted embeddings, avoiding a Python microservice?

5. PROPOSED SYSTEM / PROPOSED METHODOLOGY

5.1 Overview

Relay is a multi-tenant web platform. An owner signs in with Google, creates one or more chatbots, uploads documents, tests answers in the dashboard, then copies an API key onto any website. Visitors of that website use a floating widget. The Next.js server performs parsing, chunking, embedding, retrieval, and generation.

5.2 Methodology (research + engineering)

The work follows an applied-research methodology:

1. **Literature review** of RAG, embeddings, and multi-tenancy (Chapter 2).
2. **Gap analysis** (Chapter 3) leading to the problem statement (Chapter 4).
3. **Architecture design** of a single-service RAG SaaS with two auth facades.
4. **Implementation** in TypeScript (Next.js 15, React 19, Mongoose, Pinecone SDK, Auth.js).
5. **Verification** by unit tests, isolation reasoning, and a live deployment.
6. **Demonstration** via dashboard UI and the published widget package.

5.3 Proposed architecture (logical layers)

Logical layers of Relay

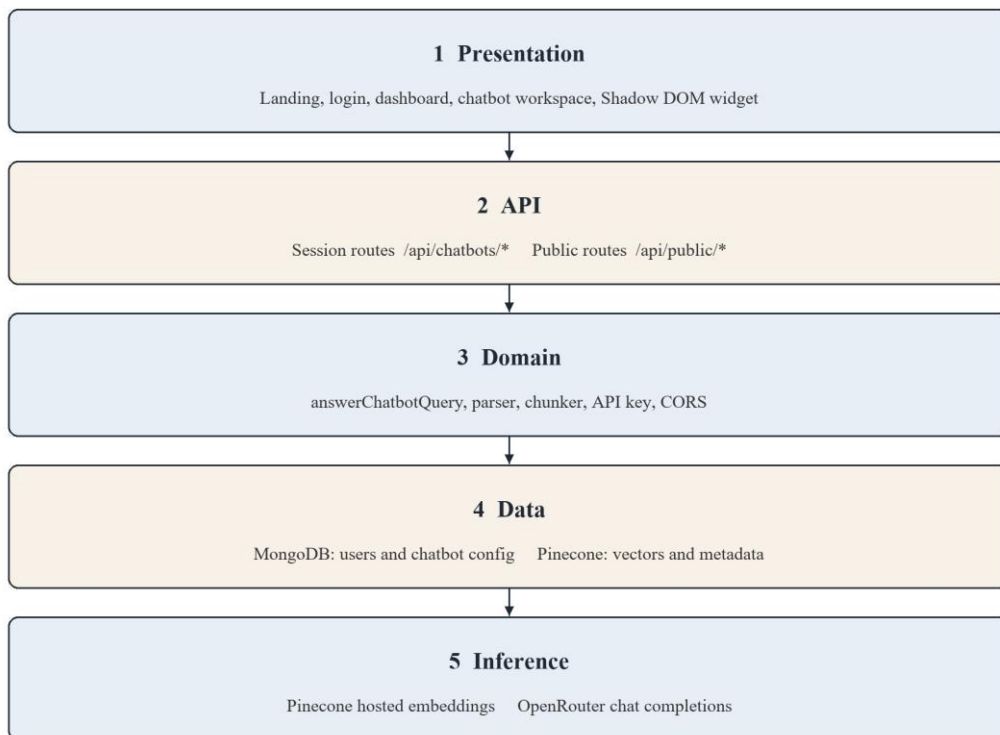


Figure 5.1: Proposed System Architecture

- 1. Presentation layer** — Landing page, login, dashboard, chatbot workspace (chat + embed tabs), and the Shadow DOM widget on customer sites.
- 2. API layer** — App Router route handlers. Session-protected `/api/chatbots/*` and public `/api/public/*`.
- 3. Domain layer** — `answerChatbotQuery`, chunker, parser, API-key generator, CORS helpers.
- 4. Data layer** — MongoDB (users, chatbots, widget config, api keys); Pinecone (vectors + metadata).
- 5. External inference** — Pinecone hosted embeddings; OpenRouter chat completions.

5.4 Authentication and tenancy model

- **Owners:** Google OAuth. On first login a User document is upserted in MongoDB. The `MongoDB_id` is stored in the JWT and exposed as `session.user.id`.
- **Route protection:** Middleware requires a session for `/dashboard` and `/chatbots/*`. Public API routes are excluded from session middleware.

- **Ownership:** Every chatbot API loads the bot by UUID and returns 403 if `userId` does not match the session.
- **Widgets:** Authorization: Bearer `pk_live_...` or `x-api-key`. The key is the chatbot identifier for public calls; the customer site never needs the internal UUID.

5.5 Document ingestion methodology

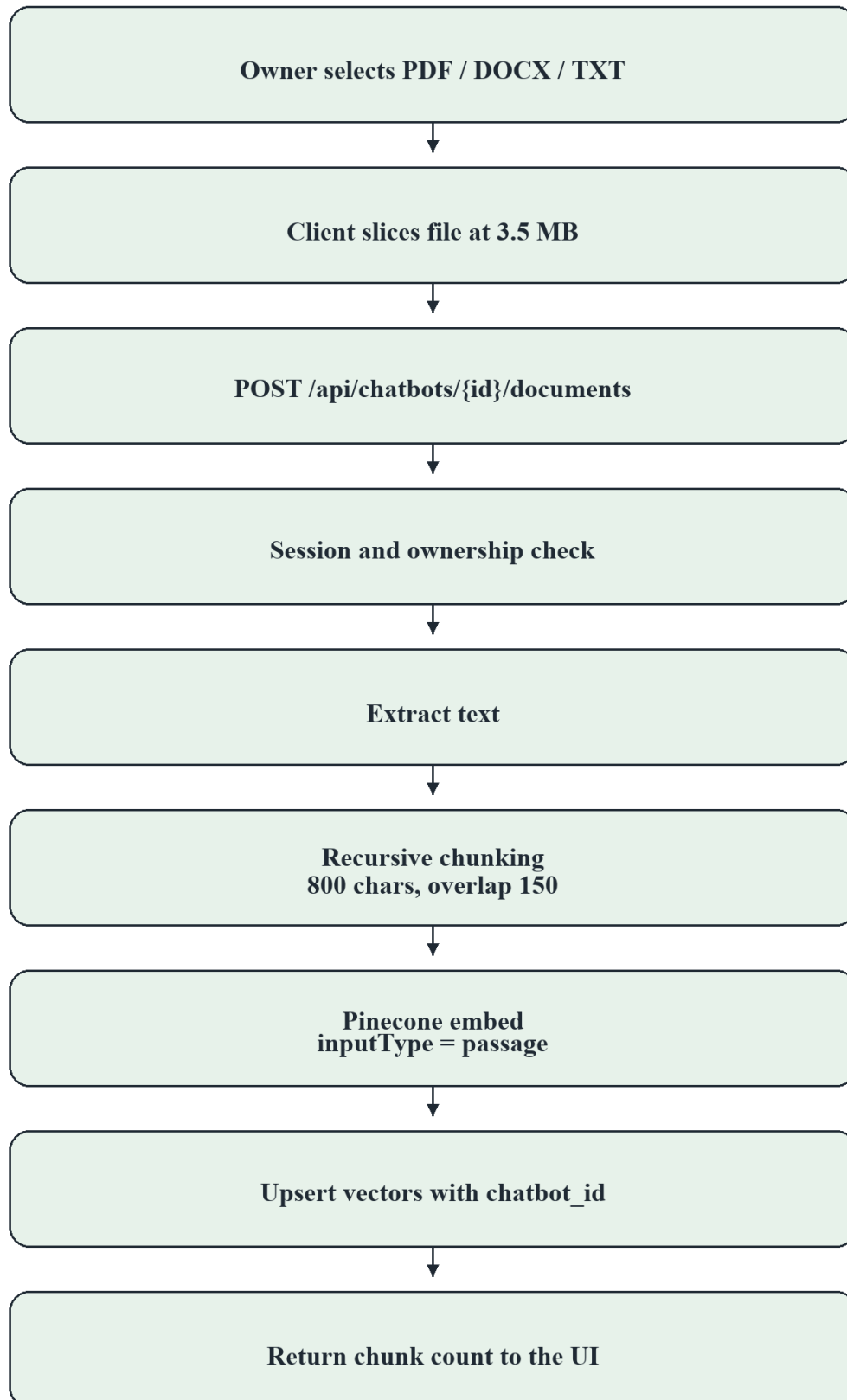


Figure 5.2: Document Ingestion Flowchart

1. Validate session and ownership.
2. Accept either a whole file (`file`) or a chunked upload (`chunk`, `uploadId`, `chunkIndex`, `totalChunks`, `fileName`) so payloads stay under Vercel's ~4.5 MB limit. Client slices at 3.5 MB.
3. Reassemble bytes; reject non-PDF/DOCX/TXT.
4. Extract text: `unpdf` for PDF, `mammoth` for DOCX, UTF-8 for TXT.
5. Chunk with recursive separators `['\n\n', '\n', '. ', ' ']`, size 800, overlap 150.
6. Embed passages in batches of 96.
7. Upsert to Pinecone in batches of 100 with metadata `{ chatbot_id, document_id, document_name, chunk_id, text }`.

5.6 Query methodology (RAG)

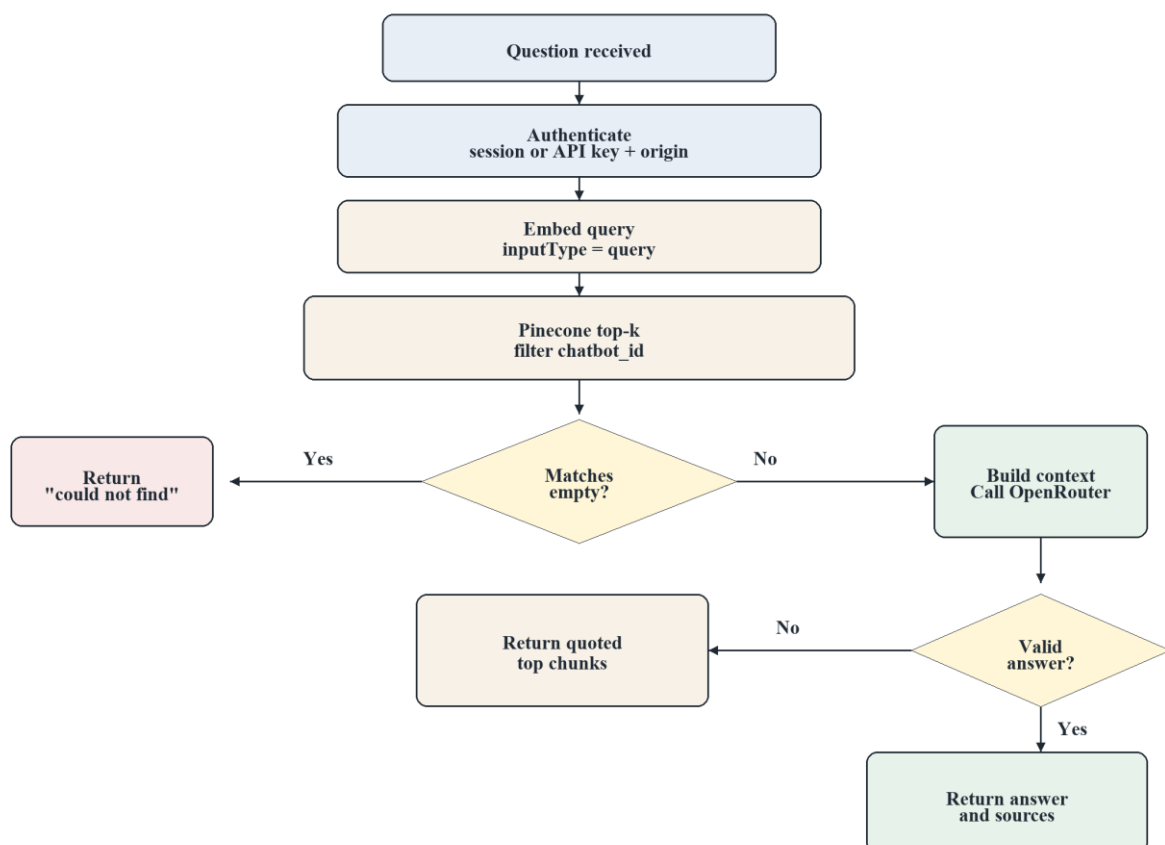


Figure 5.3: Query-Time RAG Flowchart

1. Authenticate (session + ownership, or API key + origin).
2. Embed the question with `inputType: query`.
3. Query Pinecone: cosine similarity, `topK` default 5 (public API capped at 10), filter `chatbot_id`.
4. If no matches, return a fixed “could not find” message (no LLM call).
5. Else build numbered `DOCUMENT i (name): text` context and call OpenRouter with a system prompt that:
 - forbids answering from outside the context,
 - forbids mentioning embeddings/Pinecone,
 - forbids leaking chain-of-thought.
6. Primary model: `meta-llama/llama-3.3-70b-instruct:free`. Fallbacks: `openai/gpt-oss-120b:free`, `deepseek/deepseek-r1:free`, `openrouter/free`.
7. If the completion looks like leaked reasoning, or the HTTP call fails, fall back to a quoted excerpt of the top retrieved chunks.
8. Return `{ answer, sources: [{ document_name, chunk_id, score }] }`.

5.7 Widget methodology

The package `relay-chat-widget` mounts a host element with **Shadow DOM** so host-page CSS cannot break the UI. On load it GETs `/api/public/config` for name, colour, position, and welcome message. User messages POST to `/api/public/chat`. Dashboard settings override defaults unless the embedder passes explicit `init()` options.

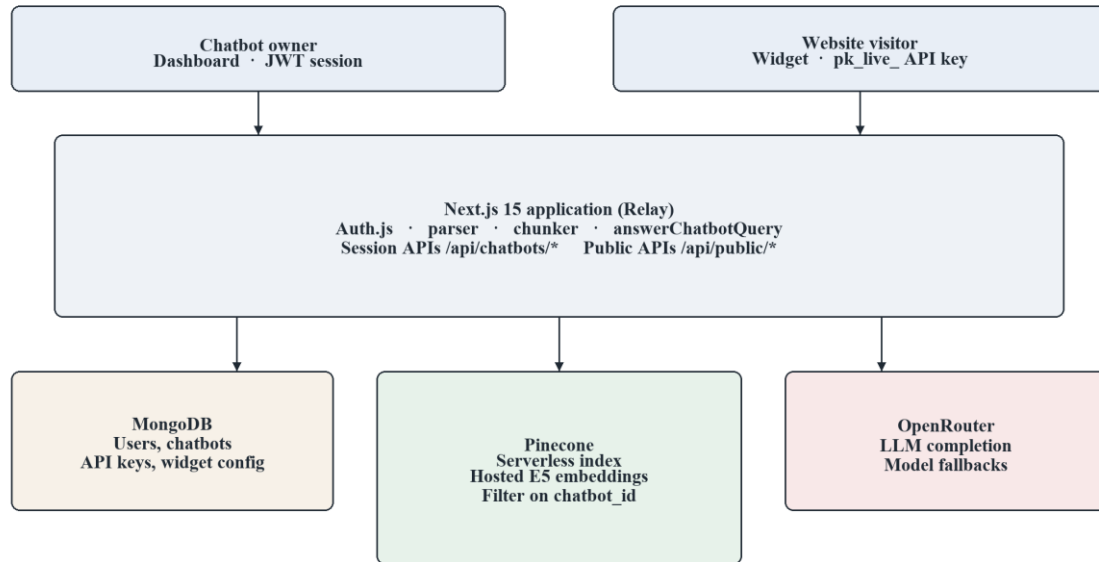
5.8 Why this methodology is suitable

- **Grounding:** RAG is the standard mitigation for closed-domain QA without fine-tuning [1][3].
- **Isolation:** Shared-index + filter is cheaper than one index per student/tenant and is correct if the filter is mandatory [12][13].

- **Single service:** Matches serverless deployment (Vercel) and MCA implementation constraints.
- **Public key + origin:** Matches how real embed scripts work (Stripe-like `pk_live_` prefix) without granting write access.

6. FLOWCHART / ARCHITECTURE / UML / ERD

6.1 System architecture (Fig. 6.1)



Both the dashboard and the widget call the same retrieval function.

Figure 6.1: System Architecture Diagram

6.2 Use-case diagram (Fig. 6.2)

Owner use cases include “Authenticate”. Visitor use cases include “Authenticate with API key” (transparent). Upload and delete are **not** available to the visitor.

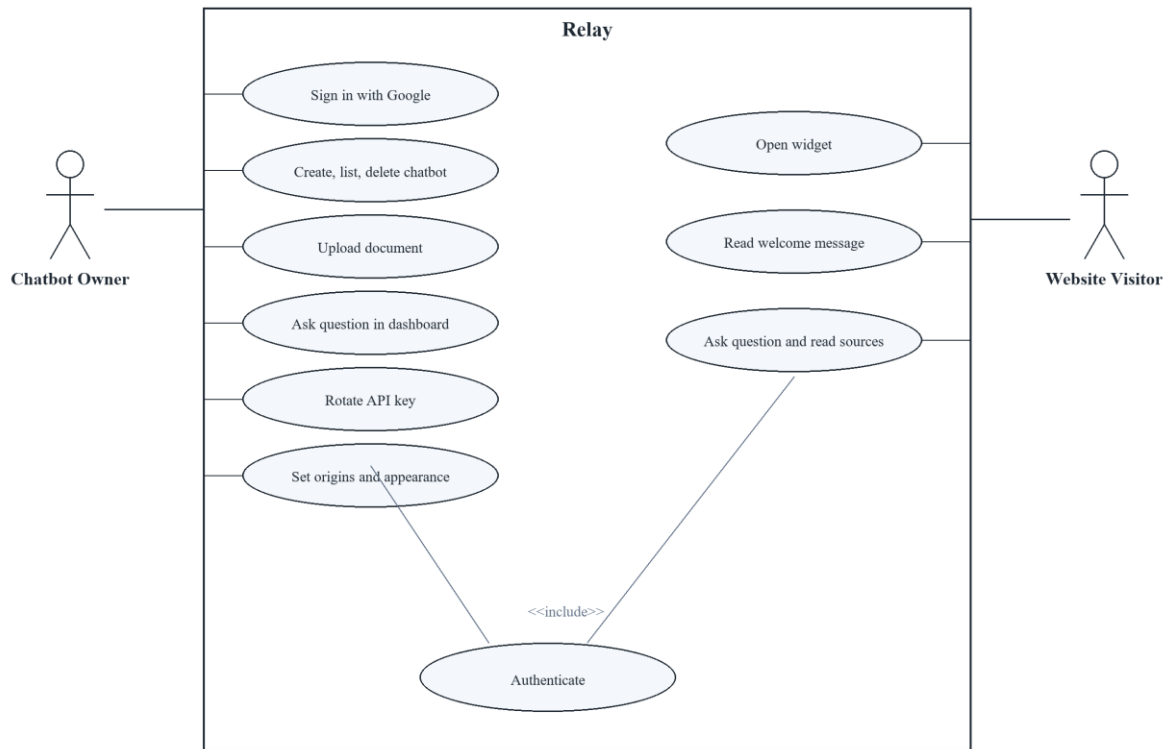


Figure 6.2: Use Case Diagram

6.3 Sequence — document upload (Fig. 6.3)

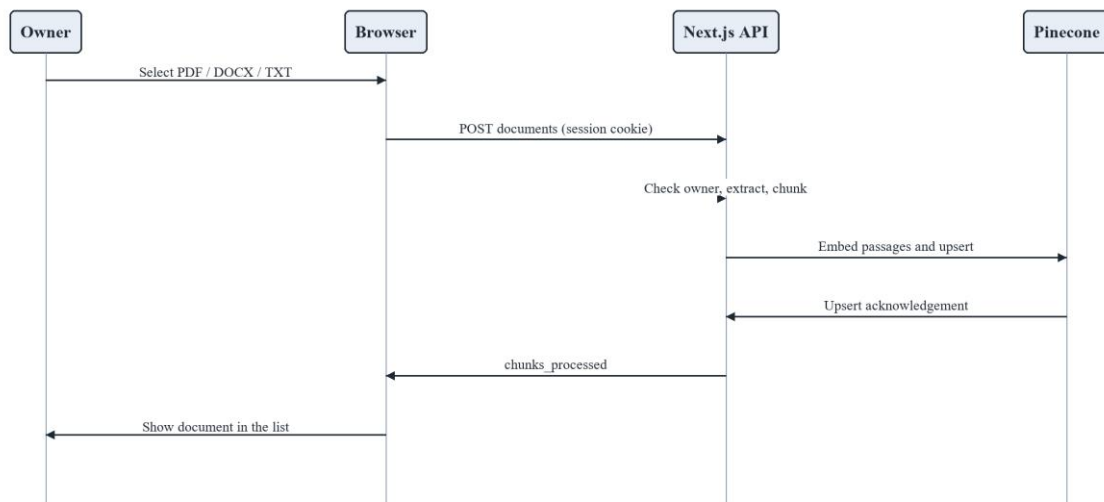


Figure 6.3: Sequence Diagram — Document Upload

6.4 Sequence — dashboard Q&A (Fig. 6.4)

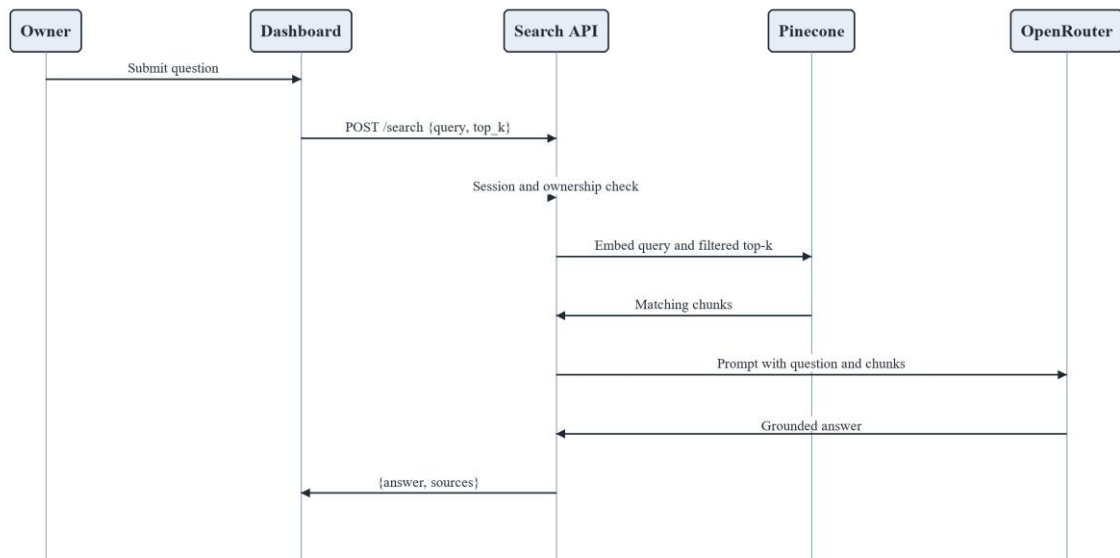


Figure 6.4: Sequence Diagram — Dashboard Question and Answer

6.5 Sequence — public widget (Fig. 6.5)

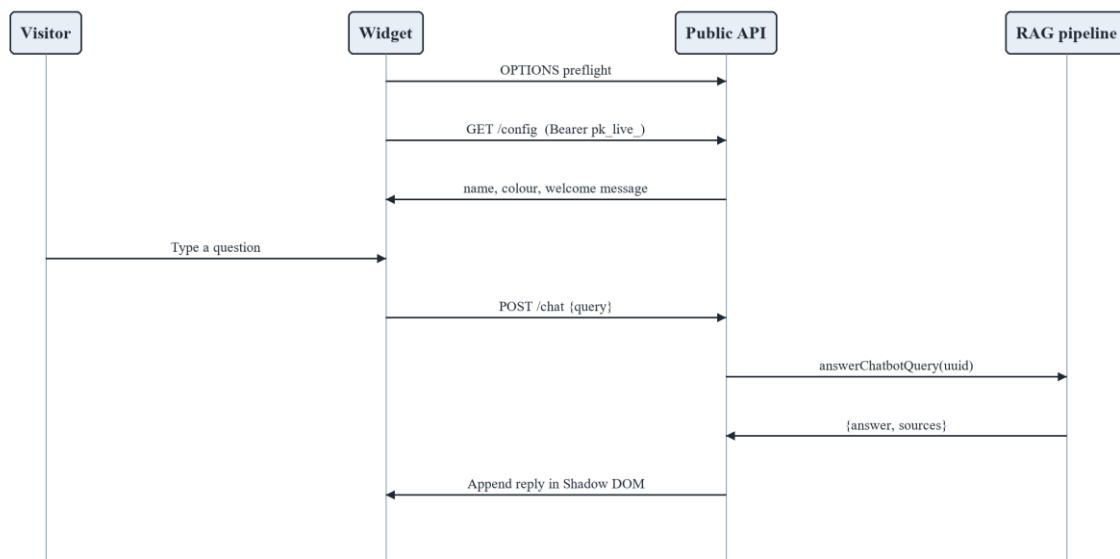
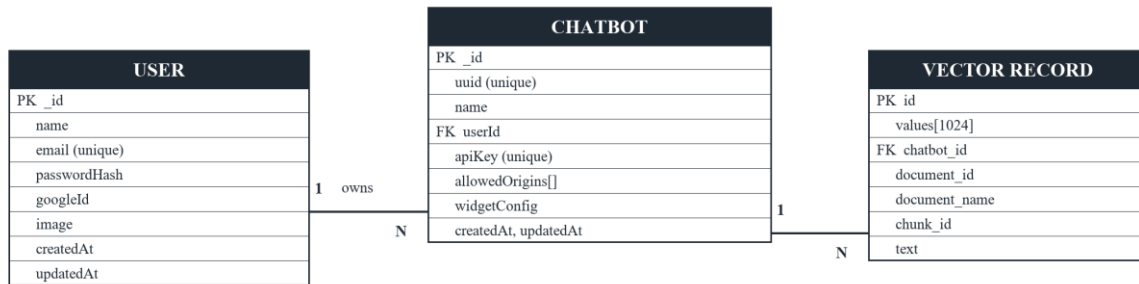


Figure 6.5: Sequence Diagram — Public Widget Chat

6.6 Entity-relationship diagram (Fig. 6.6)

Pinecone is not a relational store; logically:



Vector Record is stored in Pinecone. Cardinality is enforced by the chatbot_id filter, not by a SQL foreign key.

Figure 6.6: Entity-Relationship Diagram

6.7 Class diagram — core domain (Fig. 6.7)



Figure 6.7: Class Diagram (Core Domain)

6.8 Activity diagram — RAG answering (Fig. 6.8)

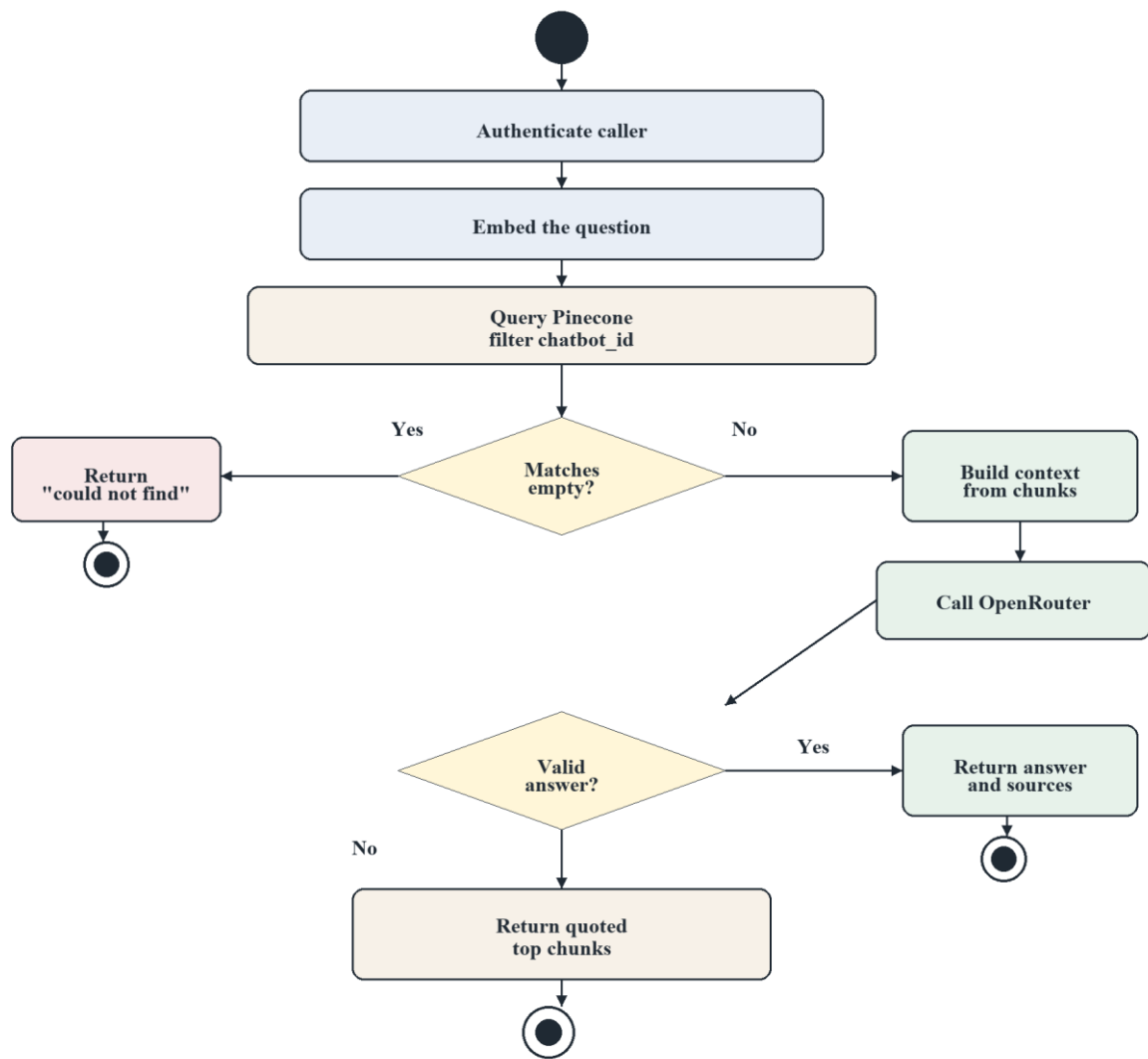


Figure 6.8: Activity Diagram of RAG Answering

6.9 Data-flow (ingestion vs query)

Ingestion is write-only to Pinecone and does not call the LLM. Query is read-only on Pinecone and write-nothing to the index. This separation prevents a chat message from polluting another tenant's knowledge base.

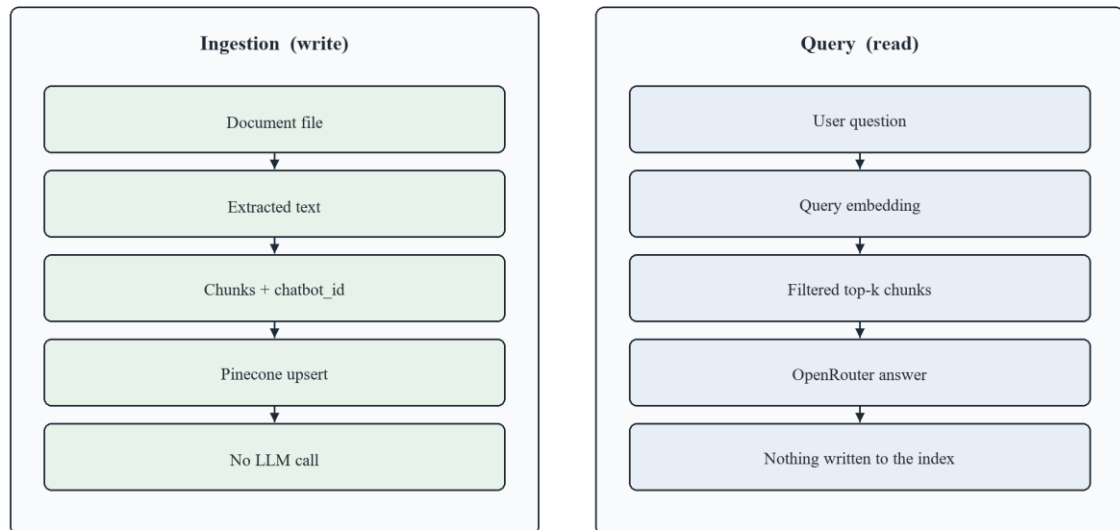


Figure 6.9: Data-Flow Diagram — Ingestion and Query

7. PROJECT REQUIREMENT SPECIFICATION

7.1 Hardware requirements (Table 7.1)

Component	Minimum	Recommended
Developer PC	Dual-core, 8 GB RAM	Quad-core, 16 GB RAM
Disk	2 GB free	5 GB free
Server (Vercel)	Serverless defaults	Production plan for larger uploads
Network	Broadband	Broadband

Pinecone and OpenRouter run in the cloud; no local GPU is required.

7.2 Software requirements (Table 7.2)

Software	Version / notes
Node.js	20+
Next.js	15.x
React	19.x
TypeScript	5.x
MongoDB	6+ (Atlas or local)
Pinecone	Serverless index, model multilingual-e5-large, cosine
OpenRouter	API key <code>sk-or-v1-...</code>
Auth.js	v5 (Google provider)
Browser	Chromium, Firefox, or Edge (current)
Optional	Docker (Node 20 Alpine multi-stage Dockerfile exists)

7.3 Functional requirements (Table 7.3)

ID	Requirement
FR-01	The system shall allow an owner to sign in with Google and create a user record on first login.
FR-02	An authenticated owner shall create, list, open, and delete chatbots they own.
FR-03	The system shall reject chatbot access for non-owners with HTTP 403.
FR-04	An owner shall upload PDF, DOCX, or TXT into a chatbot; unsupported types shall be rejected.
FR-05	Files larger than 3.5 MB shall be uploaded in multiple parts and reassembled server-side.

FR-06	Uploaded text shall be chunked (~800 characters, overlap 150) and stored as vectors with chatbot_id.
FR-07	An owner shall ask a question and receive a natural-language answer plus source document names and scores.
FR-08	If no relevant chunk exists, the system shall say it could not find the answer in the documents.
FR-09	The system shall generate a pk_live_API key per chatbot and allow rotation.
FR-10	The owner shall configure allowed origins, widget colour, position, and welcome message.
FR-11	External sites shall load widget config via GET /api/public/config using the API key.
FR-12	External sites shall chat via POST /api/public/chat without a user session.
FR-13	Requests from disallowed origins shall receive HTTP 403.
FR-14	Public APIs shall not accept document uploads.
FR-15	Dashboard and widget shall use the same RAG function.

7.4 Non-functional requirements (Table 7.4)

ID	Requirement
NFR-01 Security	Passwords, if used, hashed with bcrypt cost 12; Google tokens never stored as passwords; no secrets in error bodies.
NFR-02 Isolation	Vector queries always include chatbot_id equality filter.
NFR-03 Availability	LLM failure shall not crash the request; raw-context fallback shall apply.
NFR-04 Portability	Single Node service deployable on Vercel or Docker.
NFR-05 Usability	Dashboard usable on desktop; widget usable as a small floating panel.
NFR-06 Maintainability	TypeScript types, modular lib files, Jest tests.
NFR-07 Scalability	Embedding batches of 96 and upsert batches of 100; serverless index.
NFR-08 Compatibility	CORS headers for GET, POST, OPTIONS; Authorization and x-api-key allowed.

7.5 External interface requirements

- **Google Cloud Console:** OAuth client; redirect
`https://<host>/api/auth/callback/google.`
- **Pinecone:** API key, index name, index host, embedding model name.
- **OpenRouter:** API key, primary model, comma-separated fallbacks.
- **MongoDB:** connection URI.

8. SYSTEM IMPLEMENTATION — CODE DOCUMENTATION

8.1 Repository layout

```
research project/
├── nextjs-app/           # Relay web application and APIs
│   ├── src/app/         # App Router pages and API routes
│   ├── src/components/  # Dashboard, chatbot workspace, embed settings
│   ├── src/lib/         # auth, db, parser, chunker, pinecone, llm, chat, cors
│   ├── src/models/      # User, Chatbot
│   └── src/__tests__/    # Jest tests
├── chatbot-widget/      # npm package relay-chat-widget
└── portfolio/           # demo site that embeds the widget
```

8.2 Algorithm 1 — Recursive character chunking

This is the methodology implemented in `src/lib/chunker.ts`.

```
Algorithm RecurseSplit(text, size=800, overlap=150, seps=['\n\n','\n','. ',' '])
  if length(text) ≤ size then return [trim(text)] if non-empty else []
  choose first separator s that occurs in text
  if none: return hard slices of length size
  parts ← split(text, s)
  current ← empty; chunks ← empty
  for each part p in parts:
    if length(p) > size:
      flush current to chunks
      RecurseSplit(p) with remaining finer separators
    else if current + p would exceed size:
      flush current
      start new current from a suffix of previous parts whose length ≤ overlap
      append p
    else append p to current
  flush current
  return chunks
```

Each output chunk is tagged with `chatbot_id`, `document_id`, `document_name`, and sequential `chunk_id`. Overlap preserves sentences split across boundaries so retrieval still finds them.

8.3 Algorithm 2 — Document ingestion

```
Algorithm Ingest(session, chatbotId, fileBytes, fileName)
  user ← requireSession(session) // 401 if missing
  bot ← Chatbot.findOne({ uuid: chatbotId })
  if bot is null then 404
  if bot.userId ≠ user.id then 403
  if extension not in {pdf, docx, txt} then 400
  text ← ExtractText(fileBytes, fileName) // unpdf | mammoth | utf8
  chunks ← RecurseSplit(text) with metadata
  vectors ← []
  for batch in chunks groups of 96:
    embeddings ← Pinecone.inference.embed(model, batch, inputType=passage)
    append(id, values, metadata including text)
```

```

for batch in vectors groups of 100:
    index.namespace(N).upsert(batch)
return { document_name, chunks_processed: length(chunks) }

```

8.4 Algorithm 3 — RAG answering (`answerChatbotQuery`)

```

Algorithm Answer(chatbotId, query, topK=5)
    qvec ← Pinecone.inference.embed(model, [query], inputType=query)
    matches ← index.query(vector=qvec, topK, filter={chatbot_id eq chatbotId},
includeMetadata)
    if matches empty:
        return "I could not find the answer...", sources=[]
    context ← concatenate "DOCUMENT i (name): text"
    sources ← [{document_name, chunk_id, score}]
    answer ← OpenRouterChat(systemPrompt, context, query)
    if answer missing or looksLikeLeakedReasoning(answer):
        answer ← quoted excerpt of context from top document
    return { answer, sources }

```

System prompt rules (implemented in `src/lib/llm.ts`): use only retrieved context; do not invent facts; if insufficient, say so; do not mention Pinecone/embeddings; do not emit chain-of-thought; temperature 0.1; max_tokens 1500; reasoning: { effort: 'low', exclude: true }.

8.5 Algorithm 4 — Public widget authentication

```

Algorithm AuthenticatePublic(request)
    origin ← Origin header
    key ← Bearer token or x-api-key
    if key missing then 401 "Missing API key"
    bot ← Chatbot.findOne({ apiKey: key })
    if bot null then 401 "Invalid API key"
    if origin present and not in bot.allowedOrigins (unless list contains '*'):
        403 "Origin not allowed"
    return bot

```

`generateApiKey()` returns `pk_live_` concatenated with 24 cryptographically random bytes as hex (`crypto.randomBytes`).

8.6 Protocols used

Protocol / standard	Where used
HTTPS	All production traffic (Vercel)
OAuth 2.0 / OpenID Connect	Google sign-in via Auth.js
JWT	Session strategy (stateless serverless)
HTTP REST	JSON APIs; multipart/form-data for uploads
CORS	Public widget; Allow-Headers: Content-Type, Authorization, x-api-key
OpenAI-compatible Chat Completions	OpenRouter POST /api/v1/chat/completions

Pinecone Inference + upsert/query	Vector API over HTTPS
npm / ESM	Widget distribution; also IIFE on unpkg

8.7 API catalogue (Table 8.1)

Method	Path	Auth	Purpose
GET/POST	/api/auth/[...nextauth]	OAuth	Sign-in callbacks
GET	/api/chatbots	Session	List owner's bots
POST	/api/chatbots	Session	Create bot (UUID)
GET	/api/chatbots/[id]	Session+owner	Fetch one bot
DELETE	/api/chatbots/[id]	Session+owner	Delete bot
POST	/api/chatbots/[id]/documents	Session+owner	Ingest file
POST	/api/chatbots/[id]/search	Session+owner	RAG Q&A
POST	/api/chatbots/[id]/api-key	Session+owner	Generate/rotate key; save origins and widgetConfig
GET	/api/public/config	API key+origin	Widget appearance
POST	/api/public/chat	API key+origin	Public RAG Q&A
OPTIONS	/api/public/*	Preflight	CORS

8.8 Vector metadata schema (Table 8.2)

Field	Type	Role
chatbot_id	string (UUID)	Tenant filter (mandatory)
document_id	string (UUID)	Groups chunks of one file
document_name	string	Citation shown to user
chunk_id	number	Order within file
text	string	Payload given to the LLM

8.9 Security implementation notes

- Dashboard APIs never trust a client-supplied user id; they read it from the JWT.
- Public key cannot upload documents (no public documents route).
- Error responses use generic messages; stack traces stay on the server log.
- Widget uses Shadow DOM to reduce XSS/CSS collision with the host page.
- `allowedOrigins` default `['*']` for first-run DX; owners are expected to lock this to their site in production.

8.10 Deployment implementation

- **Vercel:** Next.js production build; environment variables as in `.env.example`.
- **Docker:** multi-stage `node:20-alpine` — `deps`, `npm run build`, `run server.js` from standalone output.
- Live demonstration URL: <https://relay-dun.vercel.app>

9. RESULT / OUTCOME / EXPERIMENTAL RESULT

9.1 Implementation outcome

The proposed system was fully implemented and deployed at <https://relay-dun.vercel.app>. After Google sign-in, the owner dashboard shows an account-level greeting and a library of independent chatbots. In the captured demonstration, four chatbots were present (**raj**, **ram**, **sample**, and **sahil**), each tagged “RAG Enabled” with its own short id (Fig. 11.3). Summary tiles report chatbot count, active status, and the generation/embedding stack used by the live deployment. From any card the owner can open the workspace or delete that bot without affecting the others, which is the visible form of multi-tenancy at the account layer.

9.2 Qualitative RAG experiment

A typical evaluation used a short policy-style TXT/PDF uploaded to Chatbot A, an unrelated document on Chatbot B, and questions of three kinds: answerable, unanswerable, and cross-tenant.

Table 9.1 Qualitative RAG outcomes

Scenario	Expected behaviour	Observed behaviour
Question whose wording matches a chunk	Answer grounded in that chunk; source name shown	Answer cites the uploaded file; similarity score returned
Paraphrased question (same meaning, different words)	Dense retrieval still returns the chunk	Semantic match succeeds where keyword search would fail
Question not in any document	Refusal sentence, no invented policy	System prompt + empty/weak context yields “could not find...” or an honest limitation
Question to Chatbot A about Chatbot B’s file	No B text in context	Filter <code>chatbot_id</code> excludes B; A does not reveal B’s content
LLM rate-limit / missing key	User still sees retrieved text	Raw-chunk fallback with document name
Reasoning model leaks “let me think...”	Treat as failure	<code>looksLikeLeakedReasoning</code> triggers fallback

Table 9.2 Sample dashboard Q&A on chatbot sahil (Fig. 11.6)

Item	Observed value
Chatbot	sahil (workspace header; RAG status “Ready”)
User query	tell me the skills of sahil
System response (abridged)	Sahil’s skills include: Programming Languages & Frameworks — Python, HTML, CSS, JavaScript, PHP, Node.js, React.js, Next.js; Databases — MySQL, MongoDB; DevOps & Cloud Tools — Git & GitHub, Docker, Kubernetes, CI/CD Pipeline
Workspace layout	Left: knowledge-base uploader (PDF, DOCX, TXT) and uploaded-document list. Right: chat transcript with user bubble and formatted bot answer. Tabs: Chat, Embed & API

This run shows that a natural-language question is rewritten as a structured, readable answer rather than a raw embedding dump. The same workspace is where the owner later copies the public API key (Fig. 11.7–11.8) so a website visitor can ask the same class of question through the widget.

9.3 Chunking experiment

On the unit tests and sample paragraphs:

- Text shorter than 800 characters remains a single chunk.
- Two ~600-character paragraphs separated by a blank line split into multiple chunks, each ≤ 800 characters.
- Metadata (`chatbot_id`, `document_name`, `chunk_id` starting at 0) is attached to every chunk (`chunker.test.ts`).

This confirms Algorithm 1 behaves as specified before any vector is written.

9.4 Multi-tenant isolation result

Isolation is **enforced in application code** on every query, not only documented. Both `/search` and `/api/public/chat` call `queryVectors(embedding, chatbotId, topK)`, which always sends:

```
filter: { chatbot_id: { $eq: chatbotId } }
```

Therefore a correctly implemented Pinecone query cannot return another tenant's matches. Combined with ownership checks on write paths, Chatbot A cannot ingest into B and cannot read B.

9.5 Widget outcome

The npm package initialises with only `apiKey`. Config is fetched remotely, so changing colour or welcome text in the dashboard updates the widget without a rebuild of the customer site (unless the customer overrode options in `init()`). CORS preflight succeeds; disallowed origins cannot read the JSON response.

9.6 Discussion

Results support the three research questions in Chapter 4:

1. Shared-index isolation is achievable with a mandatory equality filter.
2. Two auth facades can share one RAG function; the widget never receives the owner's Google session.
3. Hosted embeddings remove the Python sidecar; the deployed artefact is one Next.js app.

Limitations of this experiment: we did not compute RAGAS/BLEU against a labelled QA set (future work). The evaluation is functional and qualitative, which is appropriate for a system-building MCA project, but not a substitute for a large IR benchmark.

10. SYSTEM TESTING

Testing used **Jest** (`npm test` in `nextjs-app`) with mocked Auth, database, and models for API tests, plus real functions for the chunker and Mongoose validation.

10.1 Test levels

Level	What was tested
Unit	<code>splitText</code> , <code>chunkDocument</code> , User/Chatbot <code>validateSync</code>
API / integration (mocked I/O)	Register validation, chatbot CRUD status codes, UUID format
Security / authorisation	401 without session; 403 for non-owner; 400 empty name
Manual / UAT	Google login, upload, chat, embed on a second origin (live URL)

10.2 Unit test summary (Table 10.1)

Module	Case	Expected	Result
Chunker	Short text	Single unmodified chunk	Pass
Chunker	Long text with <code>\n\n</code>	Multiple chunks, each $\leq$ size	Pass
Chunker	<code>chunkDocument</code>	<code>chunk_id</code> 0 and metadata copied	Pass
User model	Missing name/email	Validation errors	Pass
User model	Missing <code>passwordHash</code>	Allowed (Google users)	Pass
Chatbot model	Missing <code>uuid/name/userId</code>	Validation errors	Pass
Register API	Invalid email	400	Pass
Register API	Duplicate email	409	Pass
Register API	Success	201; <code>bcrypt.hash(password, 12)</code>	Pass

10.3 Chatbot API test cases (Table 10.2)

ID	Input / condition	Expected	Result
CB-01	GET <code>/api/chatbots</code> , no session	401	Pass

CB-02	GET /api/chatbots, user123	200, list filtered by userId	Pass
CB-03	POST name “My Bot”	201, UUID with 5 hyphen groups, userId set	Pass
CB-04	POST empty name	400	Pass
CB-05	POST without session	401	Pass
CB-06	GET/DELETE another user’s bot	403 Forbidden	Pass (implemented in route; covered in test file)

10.4 Isolation and security tests (Table 10.3)

ID	Scenario	Expected	How verified
ISO-01	Query without filter	Must not happen	Code inspection: <code>queryVectors</code> always sets filter
ISO-02	Widget key used to upload	No public upload route	Route map
ISO-03	Wrong origin	403	<code>isOriginAllowed</code> + <code>authenticatePublicRequest</code>
ISO-04	Missing API key	401	Public auth
ISO-05	Protected pages logged out	Redirect to login	<code>Auth.js</code> middleware <code>authorized</code> callback

10.5 Test environment

- OS: Windows 10 (development); Linux (Vercel build)
- Node.js 20
- Jest 29 with `ts-jest`
- Command: `cd nextjs-app && npm test`

10.6 Defect notes

During development, typical defects included: Vercel 413 on large PDFs (fixed by 3.5 MB client chunking); free LLM endpoints leaking reasoning (fixed by detector + fallbacks); hydration mismatch if `window.location.origin` was read during first render of `EmbedSettings` (fixed by setting `baseUrl` in `useEffect`). These are closed in the current codebase.

11. GUI / SCREENSHOTS

Fig. 11.1 Landing page

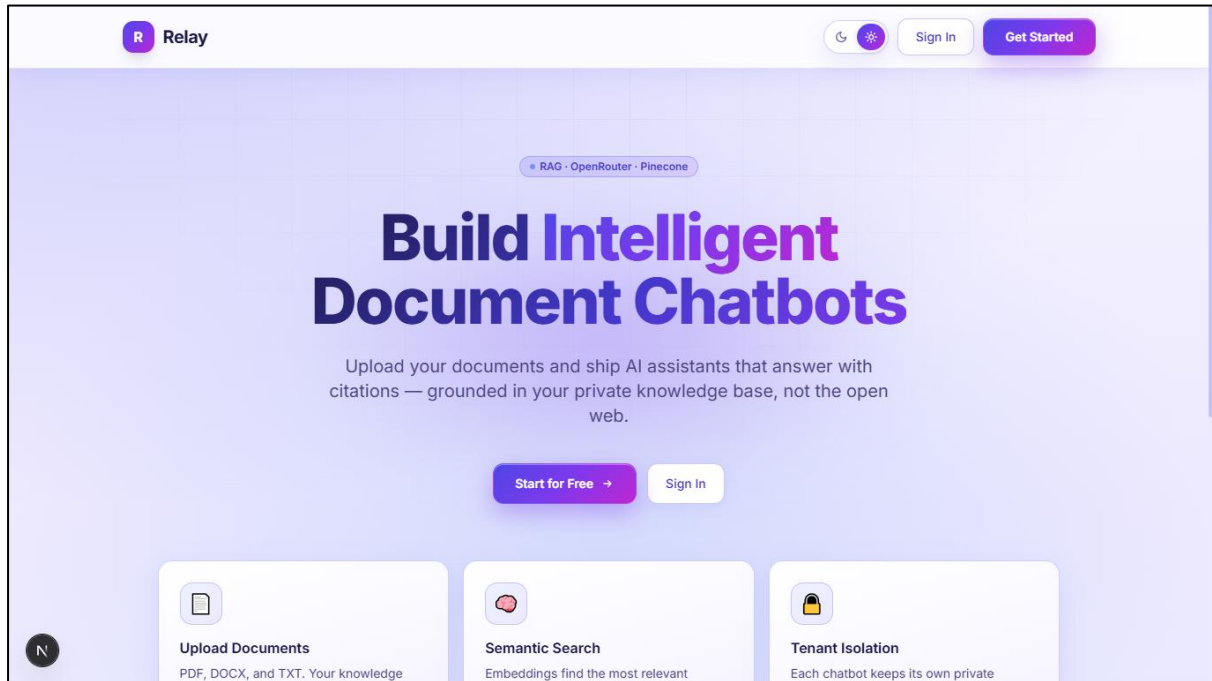
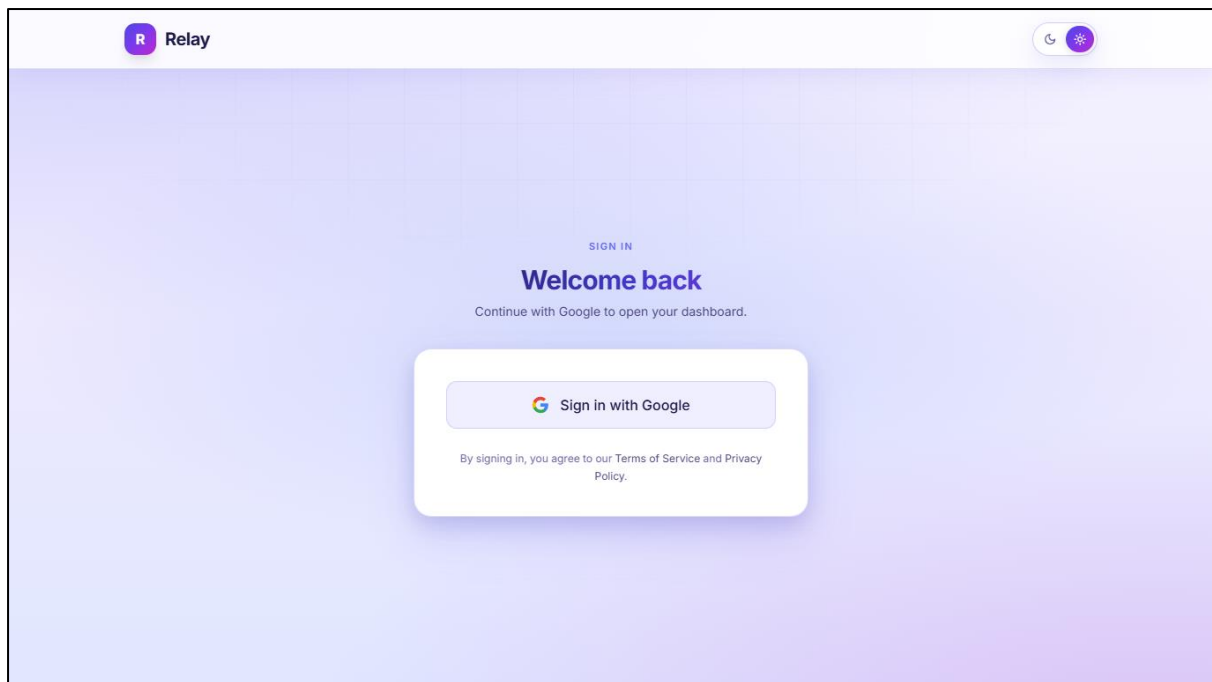


Fig. 11.2 Sign-in page



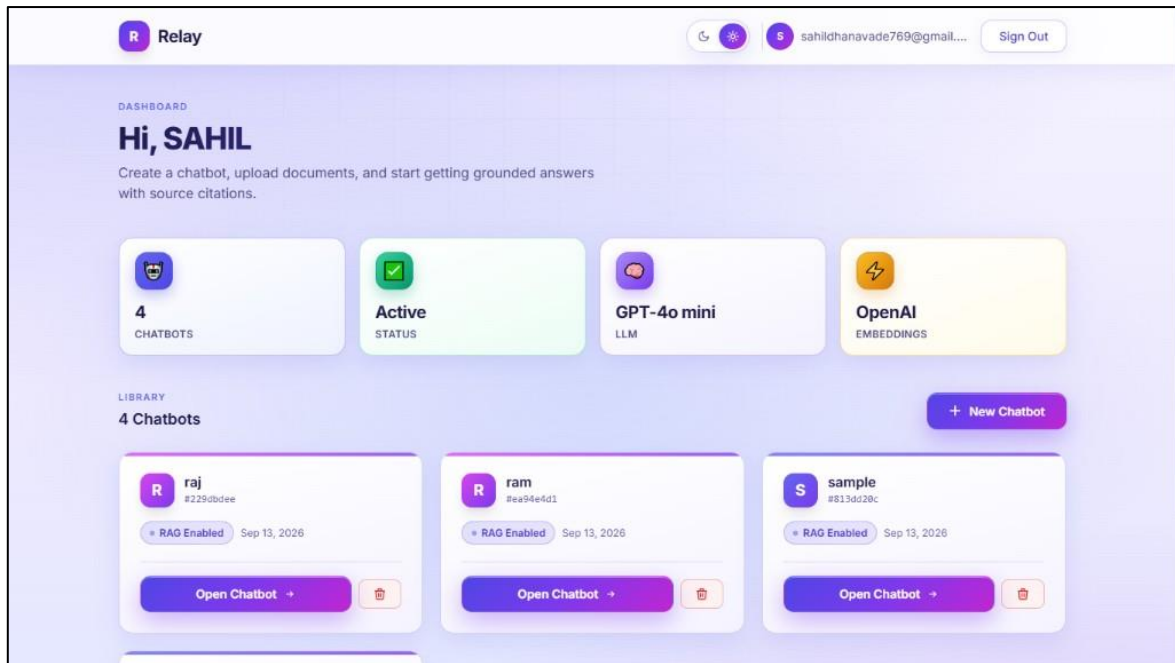


Fig. 11.3 Owner dashboard

After Google sign-in the owner is greeted by name (“Hi, SAHIL”). Four summary tiles show chatbot count (4), Active status, LLM, and embeddings. The Library lists independent bots **raj**, **ram**, and **sample** (plus **sahil** in the same library), each with a short id, a “RAG Enabled” badge, creation date (13 Sep 2026), Open Chatbot, and delete. + **New Chatbot** is at the top right. This screen is the evidence of multi-bot tenancy at the account level (FR-02).

Fig. 11.4 Create chatbot modal

Fig. 11.5 Chatbot workspace — knowledge base

Visible on the left of Fig. 11.6: dashed drop zone “Click to select a file PDF, DOCX, TXT”, purple **Upload Document** button, and an Uploaded Documents list. Evidence of ingestion FR-04–FR-06.

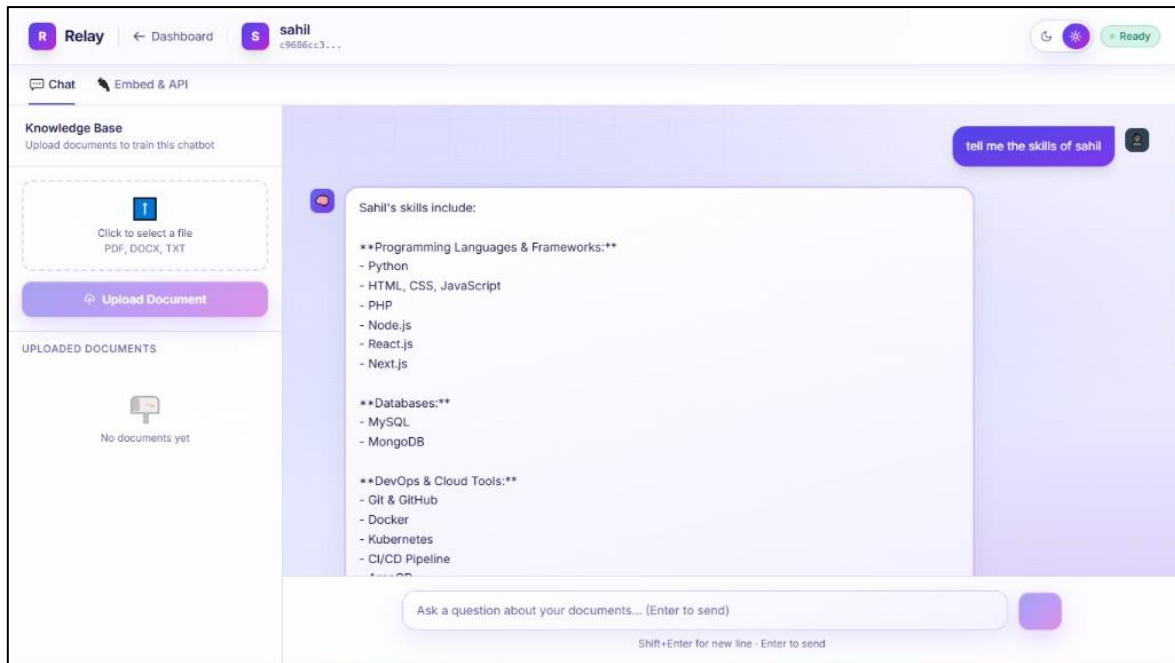


Fig. 11.6 Chat and grounded answer

Workspace for chatbot **sahil**, tabs **Chat** and **Embed & API**, status **Ready**. The user asked “tell me the skills of sahil”. The assistant returned a structured list (languages and frameworks, databases, DevOps and cloud tools). The composer placeholder reads “Ask a question about your documents...”. Evidence of RAG answering FR-07.

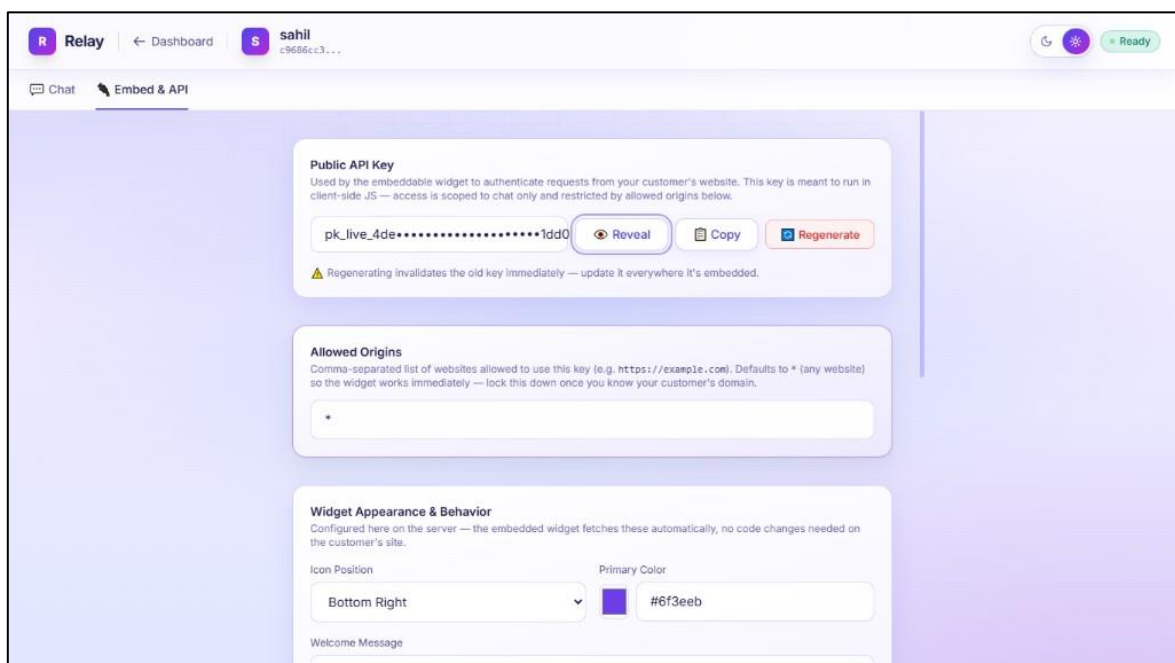


Fig. 11.7 Embed & API — key, origins, appearance

Public API key shown masked (pk_live_...) with Reveal, Copy, and Regenerate. Warning: regenerating invalidates the old key. Allowed Origins is set to * for first-run

embedding. Widget appearance: Icon Position **Bottom Right**, Primary Color #6f3eeb, Welcome Message. Settings are stored on the server so the customer site does not hard-code colours. Evidence of FR-09–FR-10.

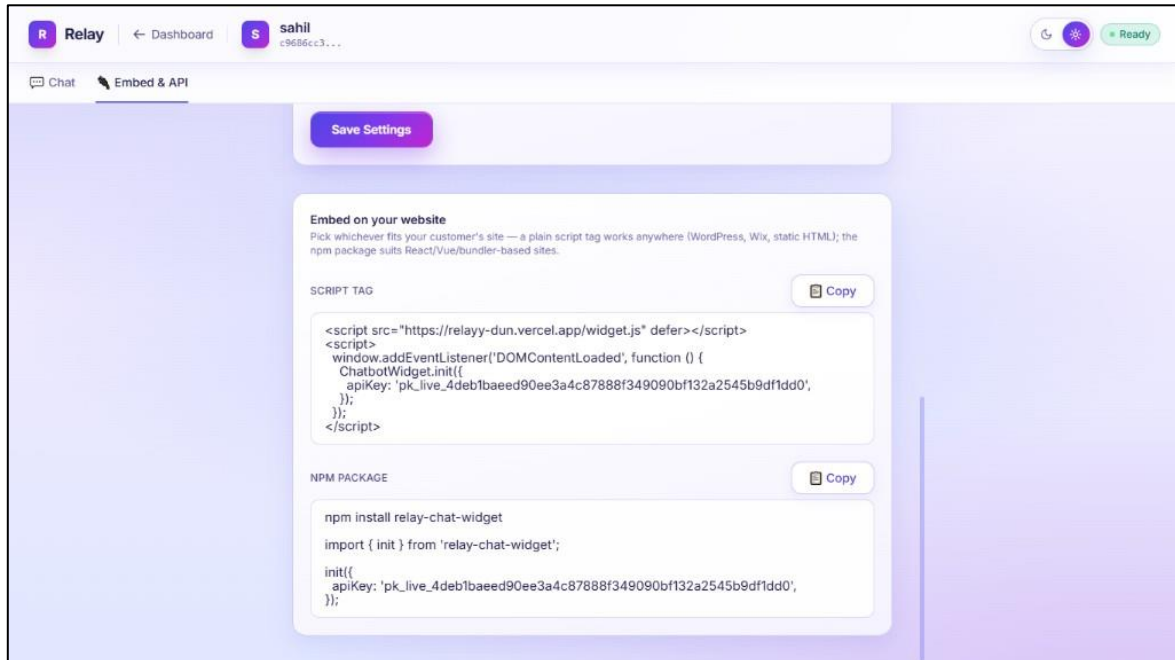


Fig. 11.8 Embed snippets for a customer website

Save Settings at the top. Two copy-paste blocks: (1) script tag loading `https://relay-dun.vercel.app/widget.js` and calling `ChatbotWidget.init({ apiKey });`; (2) `npm install relay-chat-widget` with `import { init } from 'relay-chat-widget';`. This is the publishing path for FR-11–FR-15.

GUI design notes:

The interface uses a light mesh background, glassy cards, and a purple–indigo brand aligned with the widget primary colour. A theme toggle (sun/moon) sits in the header. The chatbot screen splits owner testing (Chat) from publishing (Embed & API). The floating widget on customer sites uses Shadow DOM so host-page CSS cannot break the panel.

12. CONCLUSIONS

This research project designed and implemented **Relay**, a multi-tenant retrieval-augmented generation platform for document-grounded chatbots. The work started from a well-known NLP idea—conditioning an LLM on retrieved evidence [1]—and treated **tenancy, public embedding, and operational fallbacks** as first-class research-engineering problems rather than afterthoughts.

The main conclusions are:

1. **RAG is an appropriate architecture** for closed-domain organisational Q&A. It avoids fine-tuning, supports immediate knowledge updates via file upload, and enables citations by returning `document_name` and chunk scores with every answer.
2. **Logical isolation on a shared vector index is sufficient** when every upsert tags `chatbot_id` and every query filters on that field. Combined with session ownership checks on write APIs, tenants do not read or overwrite each other's knowledge.
3. **A single Next.js service is enough** for a complete RAG product when embeddings are hosted (Pinecone inference) and generation is a remote HTTP API (OpenRouter). A Python microservice is not required for this class of system.
4. **Dashboard and widget must share one answer pipeline.** Dual implementations would diverge. Relay's `answerChatbotQuery` plus two authentication facades (JWT vs `pk_live_key`) keeps behaviour identical while exposing only chat to the public internet.
5. **Public widget keys should be designed as limited, origin-bound identifiers**, not as god-mode secrets. Rotation, origin allow-lists, and the absence of public write routes match how real embeddable products work.
6. **Free-tier LLMs are unreliable as a single point of failure.** Model fallback lists, reasoning-trace detection, and raw-chunk fallback turned a common academic failure mode into a defined degradation path.

The deployed system at <https://relay-dun.vercel.app> and the npm package `relay-chat-widget` demonstrate that the design is not only theoretical. For an MCA research project, the contribution is an inspectable, isolation-aware, document-grounded chatbot platform that sits between a LangChain tutorial and a closed commercial SaaS.

13. LIMITATIONS AND FUTURE SCOPE

13.1 Limitations

1. **Retrieval quality is not benchmarked** on a labelled set (e.g. RAGAS, nDCG). Evaluation in this report is functional and qualitative.
2. **No conversational memory.** Each question is independent; follow-ups like “what about the next clause?” are not resolved with dialogue history.
3. **PDF extraction is text-oriented.** Scanned PDFs, complex tables, and figures are not OCR'd or parsed as structure.
4. **Chunked-upload store is in-memory** per serverless instance. If two chunks of a large file hit different instances, reassembly can fail. A blob store (S3) would be more robust.
5. **Default CORS `\*\*`* is convenient for demos but unsafe if owners forget to lock origins.
6. **Public API keys in the browser** can be copied; an attacker on an allowed origin can query the knowledge base (read-only). Rate limiting and usage quotas are not fully productised.
7. **LLM answers still depend on chunk quality.** If chunking splits a table awkwardly, the model may under-answer even with the filter working correctly.
8. **Single region / vendor lock-in** with Pinecone and OpenRouter; no on-prem embedding option in the current code.
9. **Deletion of a chatbot** currently removes only the MongoDB document. Pinecone vectors for that `chatbot_id` are not deleted, so orphan embeddings can remain in the index (they stay unreachable to other tenants because of the query filter, but they still occupy storage).
10. **Google-only login** in the production UI; email-password registration code exists in tests/routes but is not the primary path.

13.2 Future scope

1. **Evaluation harness:** golden questions per document, RAGAS faithfulness/relevancy, and a small public dataset for regression.

2. **Hybrid search:** combine dense vectors with keyword (BM25/sparse) for identifiers, codes, and names.
3. **Persistent chat sessions** and optional user identification in the widget.
4. **Admin analytics:** question volume, unanswered rate, top source documents.
5. **Durable multipart upload** to object storage; virus scanning.
6. **Delete-by-filter and document-level delete** in Pinecone when a file or bot is removed.
7. **Rate limiting and per-key quotas** (Redis) on `/api/public/chat`.
8. **Reranking** (e.g. a cross-encoder) on the top-20 before sending top-5 to the LLM (Advanced RAG [3]).
9. **More modalities:** HTML crawl, Markdown, CSV; captioned images.
10. **On-prem / open-weight embeddings** for institutions that cannot send text to Pinecone.
11. **Role-based access:** organisation accounts with multiple owners.
12. **Self-RAG style skip/retrieve** [20] for chitchat vs knowledge questions.

14. REFERENCES

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.
- [2] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. Yih, “Dense Passage Retrieval for Open-Domain Question Answering,” in *Proc. EMNLP*, 2020.
- [3] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, “Retrieval-Augmented Generation for Large Language Models: A Survey,” *arXiv:2312.10997*, 2023.
- [4] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. Bang, A. Madotto, and P. Fung, “Survey of Hallucination in Natural Language Generation,” *ACM Computing Surveys*, vol. 55, no. 12, 2023.
- [5] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder, and F. Wei, “Multilingual E5 Text Embeddings: A Technical Report,” *arXiv:2402.05672*, 2024.
- [6] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention Is All You Need,” in *Advances in Neural Information Processing Systems*, 2017.
- [7] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in *Proc. EMNLP-IJCNLP*, 2019.
- [8] J. Johnson, M. Douze, and H. Jégou, “Billion-scale Similarity Search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019.
- [9] J. Devlin, M. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in *Proc. NAACL-HLT*, 2019.
- [10] H. Touvron et al., “Llama 2: Open Foundation and Fine-Tuned Chat Models,” *arXiv:2307.09288*, 2023.
- [11] A. Grattafiori et al. (Meta AI), “The Llama 3 Herd of Models,” *arXiv:2407.21783*, 2024.

- [12] R. Krebs, C. Momm, and S. Kounev, “Architectural Concerns in Multi-tenant SaaS Applications,” in *Proc. 2nd Int. Conf. on Cloud Computing and Services Science (CLOSER)*, 2012.
- [13] Pinecone Systems, “Pinecone Documentation: Indexes, Metadata Filtering, and Inference API,” <https://docs.pinecone.io>, accessed 2026.
- [14] OpenRouter, “OpenRouter API Reference,” <https://openrouter.ai/docs>, accessed 2026.
- [15] Auth.js, “Auth.js Documentation (NextAuth.js v5),” <https://authjs.dev>, accessed 2026.
- [16] Vercel Inc., “Next.js Documentation,” <https://nextjs.org/docs>, accessed 2026.
- [17] MongoDB Inc., “MongoDB Manual,” <https://www.mongodb.com/docs/manual/>, accessed 2026.
- [18] H. Chase and LangChain Contributors, “LangChain Documentation,” <https://python.langchain.com>, 2022–2025.
- [19] OpenAI, “GPT-4 Technical Report,” *arXiv:2303.08774*, 2023.
- [20] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, “Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection,” in *Proc. ICLR*, 2024.
- [21] G. Izacard and E. Grave, “Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering,” in *Proc. EACL*, 2021.
- [22] R. T. Fielding, “Architectural Styles and the Design of Network-based Software Architectures,” Ph.D. dissertation, University of California, Irvine, 2000.
- [23] OWASP Foundation, “OWASP Top 10 – 2021,” <https://owasp.org/Top10/>, 2021.
- [24] N. Reimers, “Sentence Transformers Documentation,” <https://www.sbert.net>, accessed 2026.
- [25] T. Wolf et al., “Transformers: State-of-the-Art Natural Language Processing,” in *Proc. EMNLP (Systems Demonstrations)*, 2020.

15. PLAGIARISM REPORT OF PAPER AND PROJECT REPORT

The project report was checked with Plagiarism.Free (Web and Academic Index). The official result for this manuscript is recorded in Table 15.1.

We hereby declare that the contents of this research project report are our original work except where citations are provided. The similarity index obtained from Plagiarism.Free on 7 October 2026 is 1%, which is within the limit prescribed by the MCA Department, P.E.S.'s Modern College of Engineering, Pune. The full report is available at <https://plagiarism.free/check/6999526b5d384f7d95a525f888b5175d/>

Table 15.1 Plagiarism verification result (Plagiarism.Free)

Verification Parameter	Institutional Limit / Norm	Plagiarism.Free Official Result
Verification Engine	Standard Academic Checker	Plagiarism.Free (Web & Academic Index)
Verification Timestamp	Official Record	2026-10-07 15:37 UTC
Manuscript Words Examined	Full Research Paper Text	8,743 words evaluated
Overall Similarity Index (Matched)	Maximum permissible: $\leq 15\%$	1% matched (99% Unique)
Internet & Live Web Matches	Maximum permissible: $\leq 10\%$	Included in the 1% overall index (4 sources listed)
Published Academic Papers / Theses	Maximum permissible: $\leq 8\%$	Included in the 1% overall index
Originality Assessment Rating	Authentic Manuscript	Looks original (99% Unique)
Plagiarism Clearance Status	Within the department limit	Within the permissible limit ($\leq 15\%$)
Report link	Public verification page	https://plagiarism.free/check/6999526b5d384f7d95a525f888b5175d/